
Service-based Hybrid Intelligence Software Systems

Hong-Linh Truong
linh.truong@aalto.fi



02.09.2026



Content is available under
CC BY-SA 4.0 unless otherwise stated

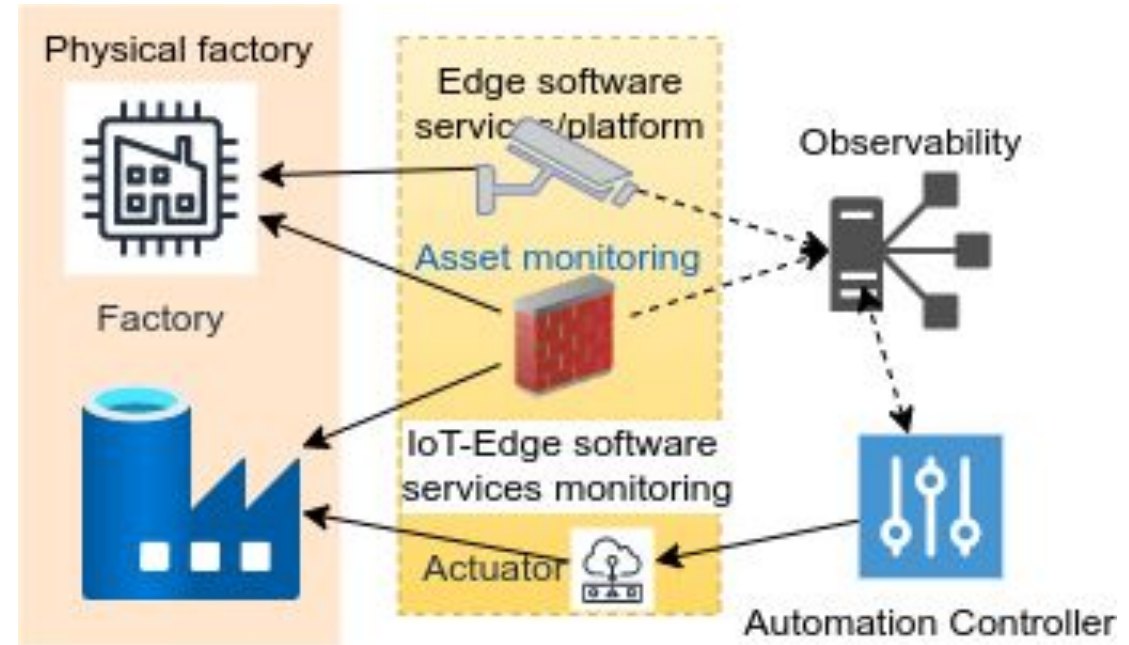
Learning objectives

- Understand potential scenarios and applications of hybrid intelligence software
- Understand and apply diverse types of service-based capabilities in continuum computing
- Understand hybrid intelligence software (HIS)
- Understand and analyze key non-functional requirements for HIS w.r.t. diversity, sovereignty and compliance

Content

- Scenarios
- Computing continuum as AI-intensive infrastructure enablers
- Composability: capabilities and components
- Hybrid intelligence software: intelligence continuum, quality of analytics, and requirements for diversity, sovereignty and compliance

Security Management Service



- **Different types of capabilities**

- IoT sensing/monitoring, data analytics, conventional ML inferences, etc.
- for analysis, optimization and control functionalities
- automated tasks

- **Autonomy is highly expected**

What types of intelligence do we need for the software powered the service? Do we need different types of intelligence?

Security Management Service:

as one example of hybrid intelligence software

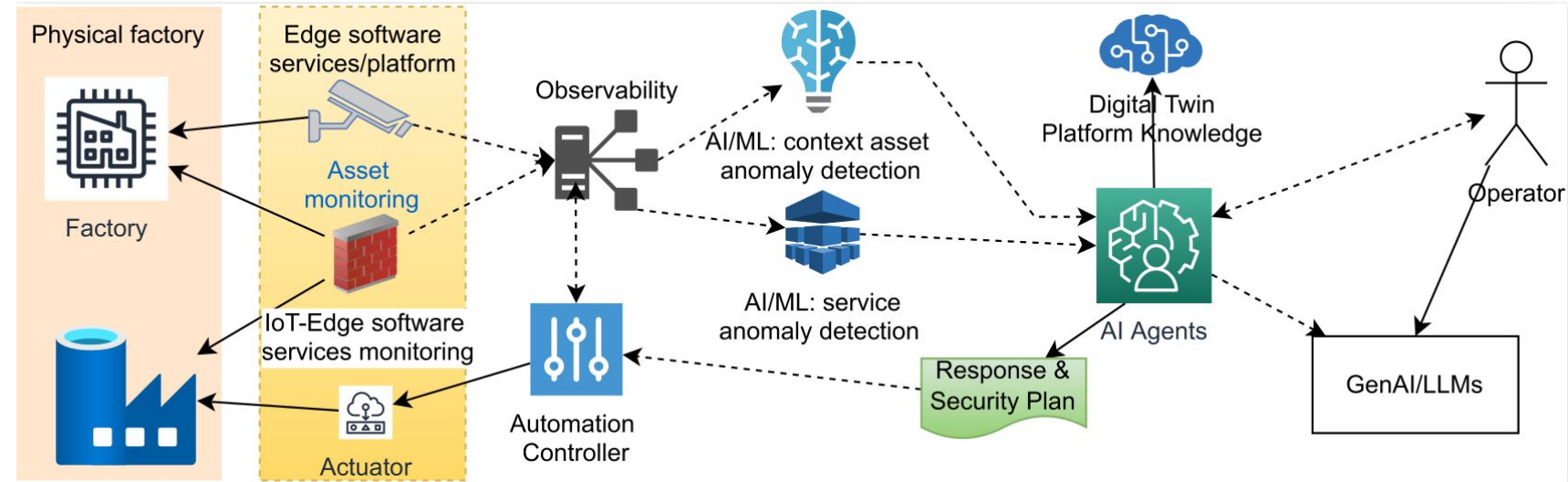


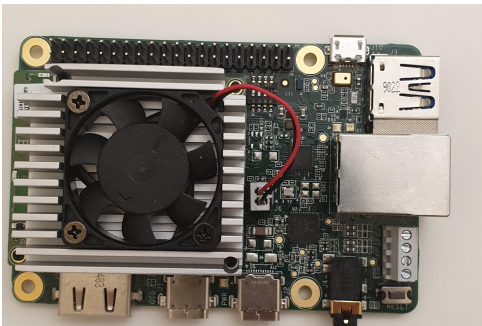
Figure source: Hong-Linh Truong, "New Frontiers in Service Engineering Analytics for Multi-Continuum Systems (FrontSEA)", working paper, 2025

- **Compositions of AI-Human**
 - capabilities from GenAI/LLMs, AI Agents, and humans
 - replacement and combination techniques for intelligence provisioning
- **Capabilities span multiple administrative, distributed domains**
 - in edge and cloud, from multiple providers

Computing Continuum as AI-intensive infrastructure enablers

Edge computing: enable computing capabilities at the edge

Coral with Edge TPU
System-on-Module,
Google Edge TPU ML
accelerator
coprocessor



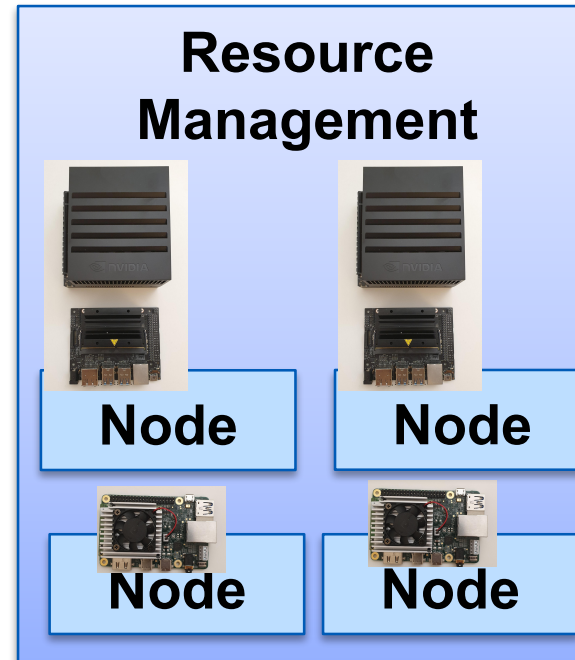
Jetson NVIDIA
(GPU+CPU)



NVIDIA DGX Spark



Edge systems



Our testbed for
this course



Cloud computing/HPC: powerful and heterogeneous computing capabilities in (public and enterprise) data centers

- Clusters of VMs/containers
 - e.g., in Aalto we use Triton (<https://scicomp.aalto.fi/triton/>) and CSC (<https://www.csc.fi/>)
- High performance systems
 - large-scale
 - known accelerators
 - GPU and FPGA
 - High-bandwidth, low-latency interconnect fabrics for communication among CPU/GPU, memory and storage

Google Cloud

Azure

amazon
web services™

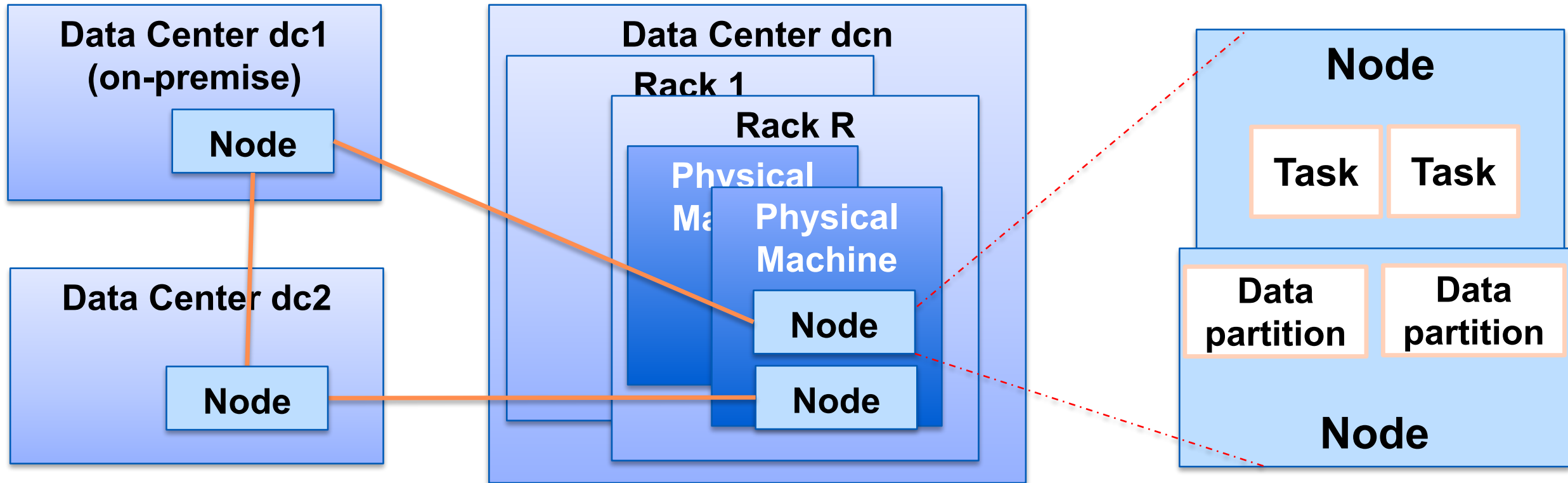


Illustration of the HPE Cray EX cabinets. Copyright: Hewlett Packard Enterprise

Figure source:

<https://www.lumi-supercomputer.eu/deep-dive-into-the-building-of-the-lumi-data-center/>

Large-scale distributed infrastructures for multi-tenant, big data/AI applications

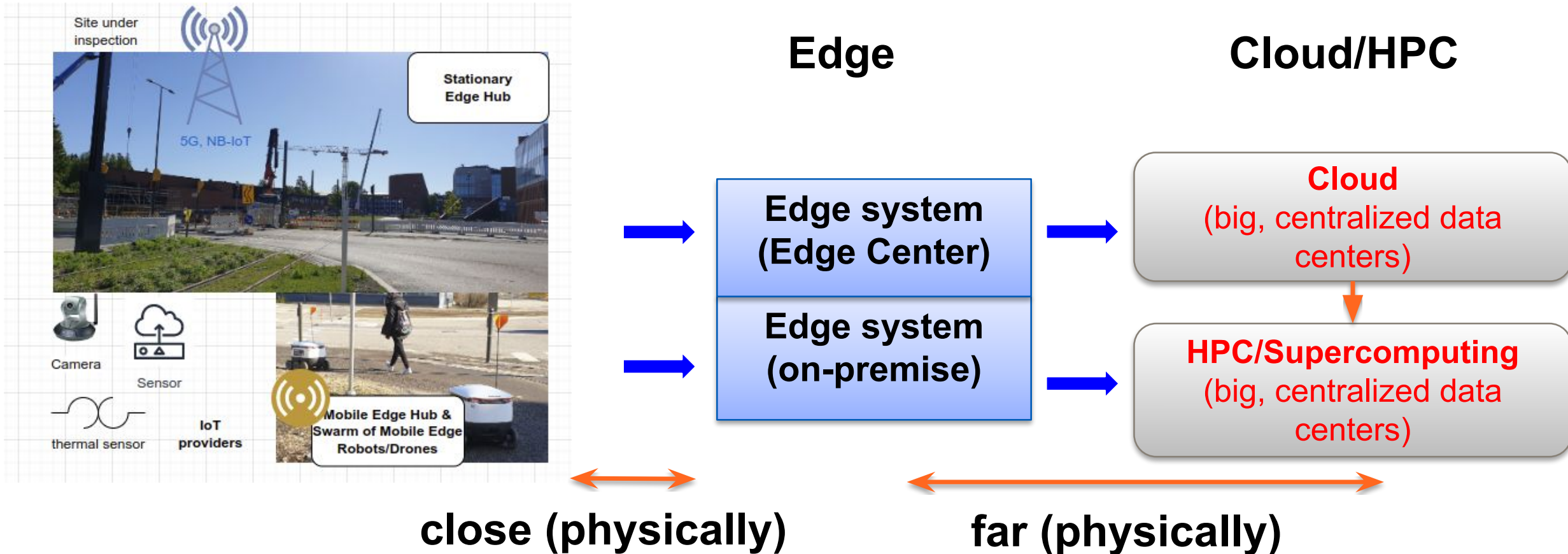


Data centers: cloud data centers, edge data centers, HPC centers

Edge-cloud continuum

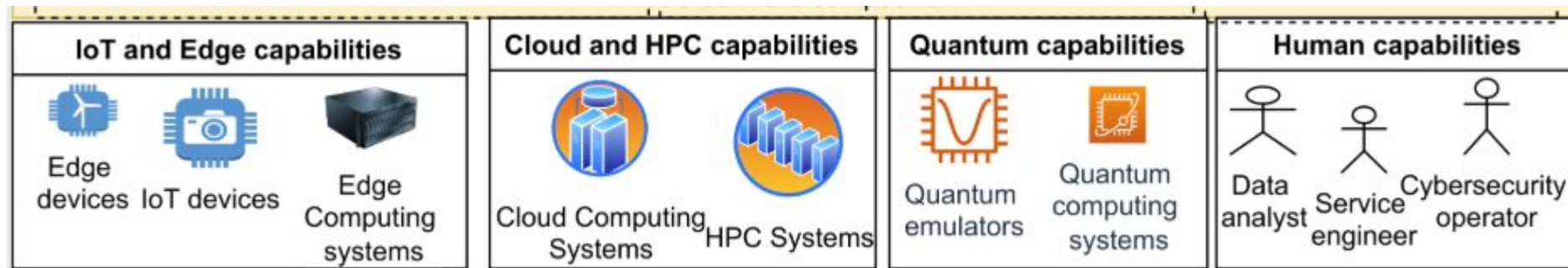
- Interconnecting edge and cloud resources and using them in similar manners
 - place and move tasks seamlessly between edges and clouds like in the same system
 - reduce issues and costs due to diverse development, deployment and operations
- From the technical viewpoint: for both edge and cloud
 - use similar enabling technologies, like containers and orchestrators
 - employ similar management techniques and methods (deployment, monitoring, policies, etc.)
- Enable widely decentralized complex workloads
 - AI/ML, data analytics, and real-time operations in distributed and heterogeneous environments

Edge-cloud-HPC continuum



Capabilities from edge, cloud, hpc, and quantum

- Multiple types of capabilities are available for software/service composition and integration
- AI-intensive software relies on diverse types of capabilities in edge-cloud continuum and from humans

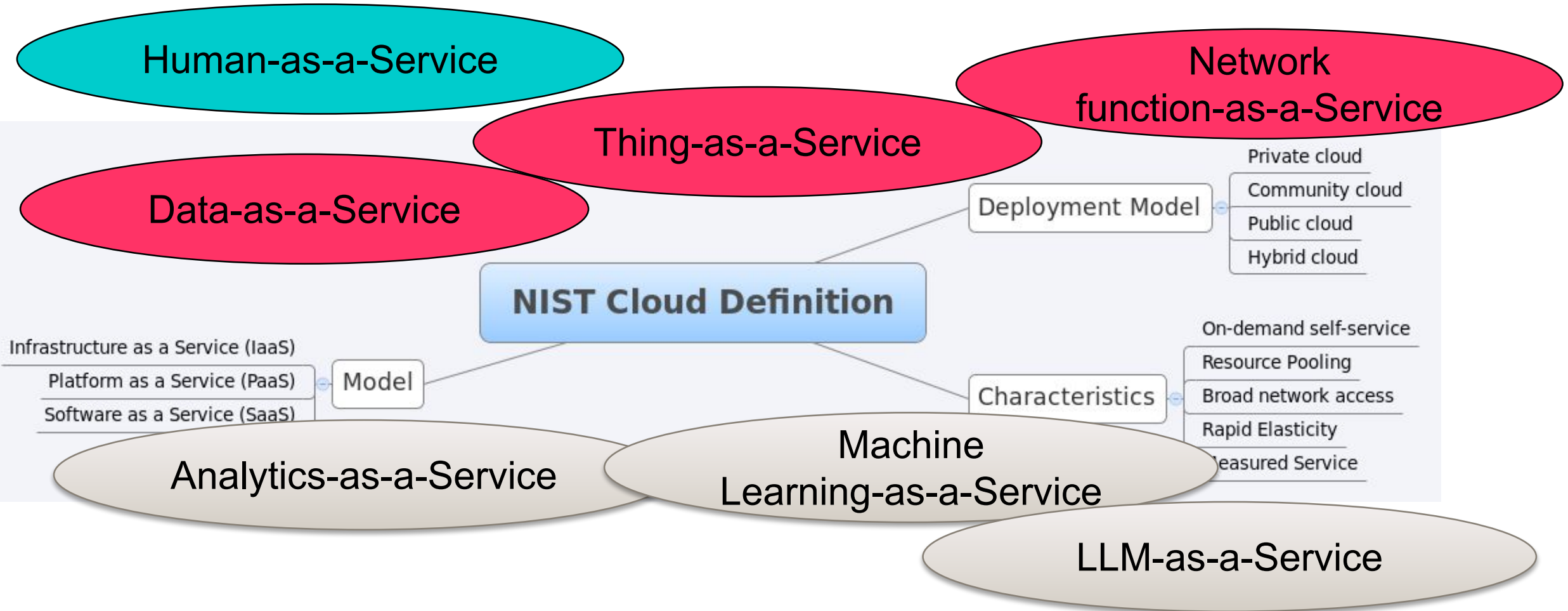


Services as a delivery model for diverse types of capabilities

Services

- Service
 - reusable, independent software components, which can be accessed remotely with well-defined interfaces/APIs via standard protocols
 - e.g., REST, RPC, and AMQP
 - Microservices as one popular design and implementation approach
- Service-based system
 - consists of multiple components; components are services
 - provisioned and deployed as services
- Separation from infrastructures/resource layers
 - resources can be provided by infrastructure services: computing, storage, memory, and connectivities
 - e.g., a drone can be provided as a resource for AI-based sensing

Types of Services



NIST definition: <https://nvlpubs.nist.gov/nistpubs/legacy/sp/nistspecialpublication800-145.pdf>

Microservices as one design and implementation approach

- Many components for data storage, data processing, ML inferences, controlling, etc.
 - act as basic elements of a complex system
- Acting as an intermediary
 - providing access to large (service) complex systems (e.g., to LLMs and LLM serving)

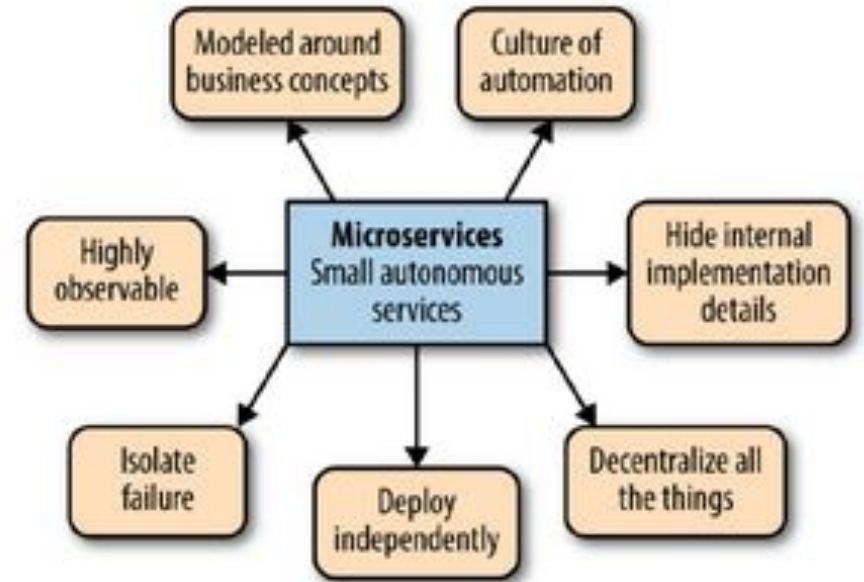
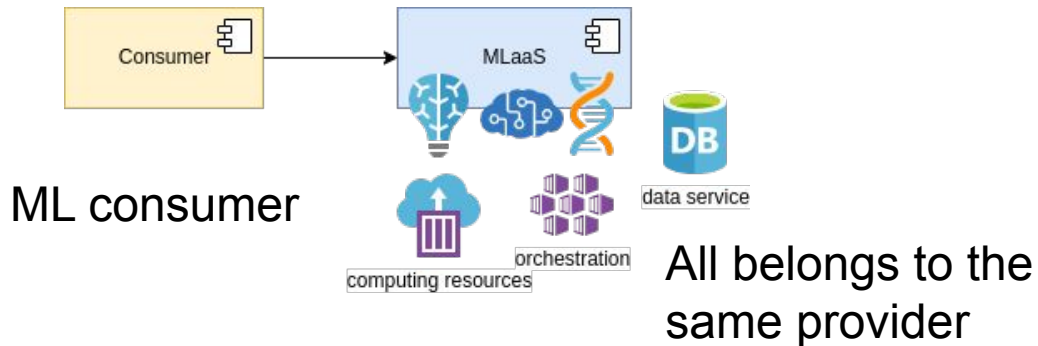


Figure source: Sam Newman, Building Microservices, 2015

Service consumer and provider models: Service coupling and deployment and service contracts

Two stakeholders engagement (Google, Microsoft, OpenAI, etc.)

Example in ML as a Service:

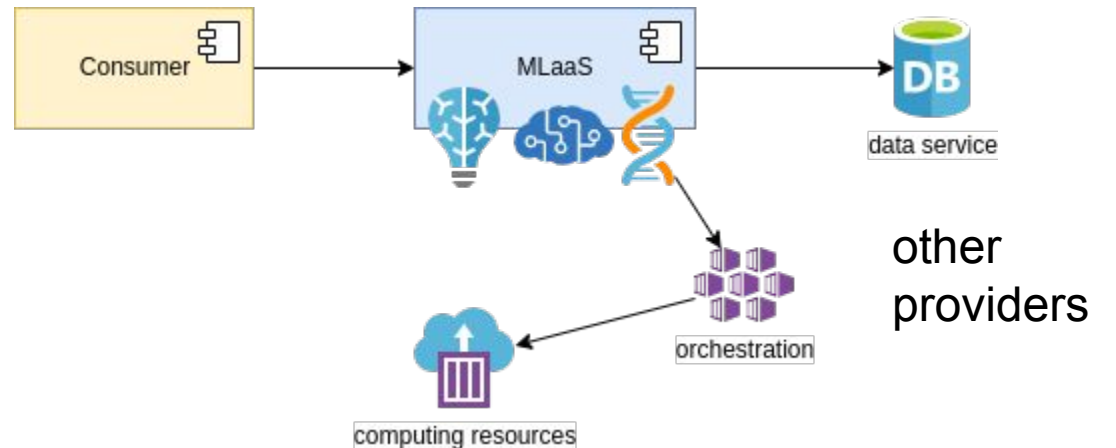


A special form: for internal uses in a large company

A!

Three stakeholders engagement

Example in ML as a Service:



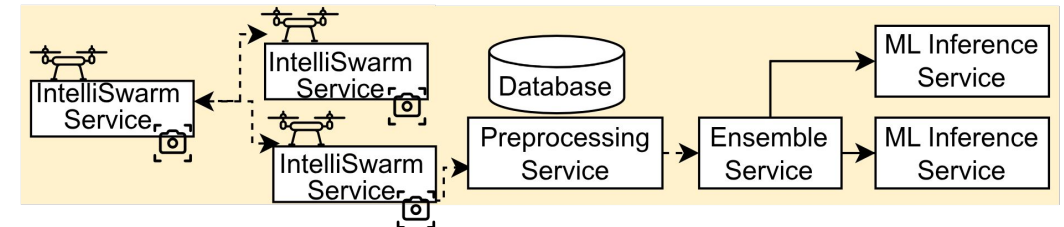
Relies on external platforms for orchestration, computing resources and other data (e.g., using Sedon, SagerML, etc.)

Service consumer and provider models: complex supply chains

Complex software supply chains and stakeholders in development and operations:

- data, AI/ML models, GenAI/LLM services and various services
- computing resources from swarm-edge-cloud continuum
- intelligence capabilities from AI/GenAI/ML and human capabilities (in-house vs external)

Example: for a swarm system



Service Provider:

drone services, computing continuum, AI/ML inference platforms

Application provider:

sensing services, AI/ML training and fine-tuning, AI/ML serving, pipelines

Third-party providers:

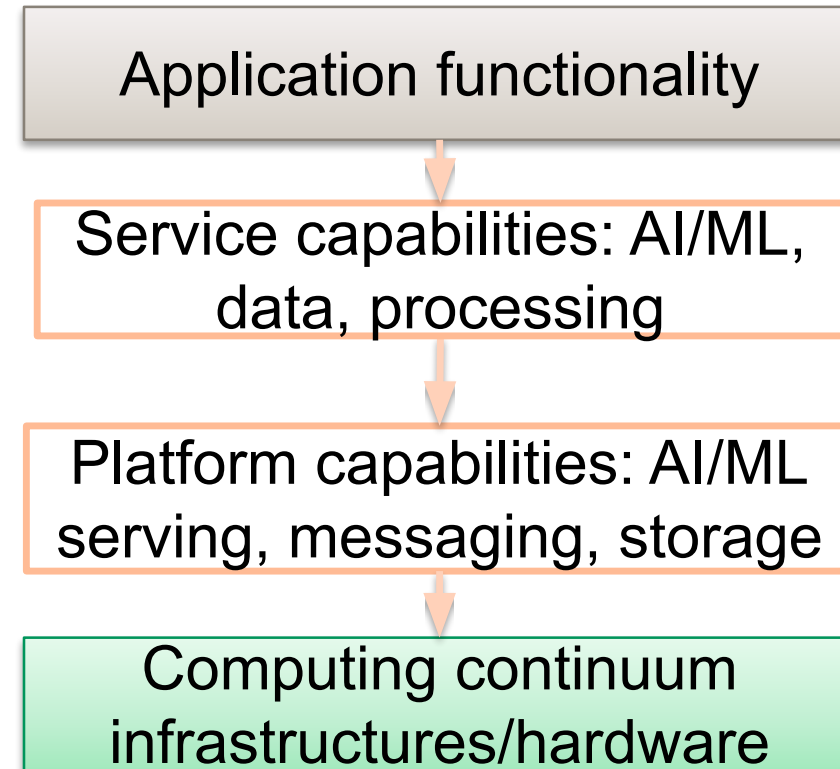
training data, libraries, ML/foundation models, domain knowledge, programming frameworks, utilities, ...

Key challenges: diversity, sovereignty and compliance

Service composability: capabilities and components

Common layers

Which components do what, and where are they?

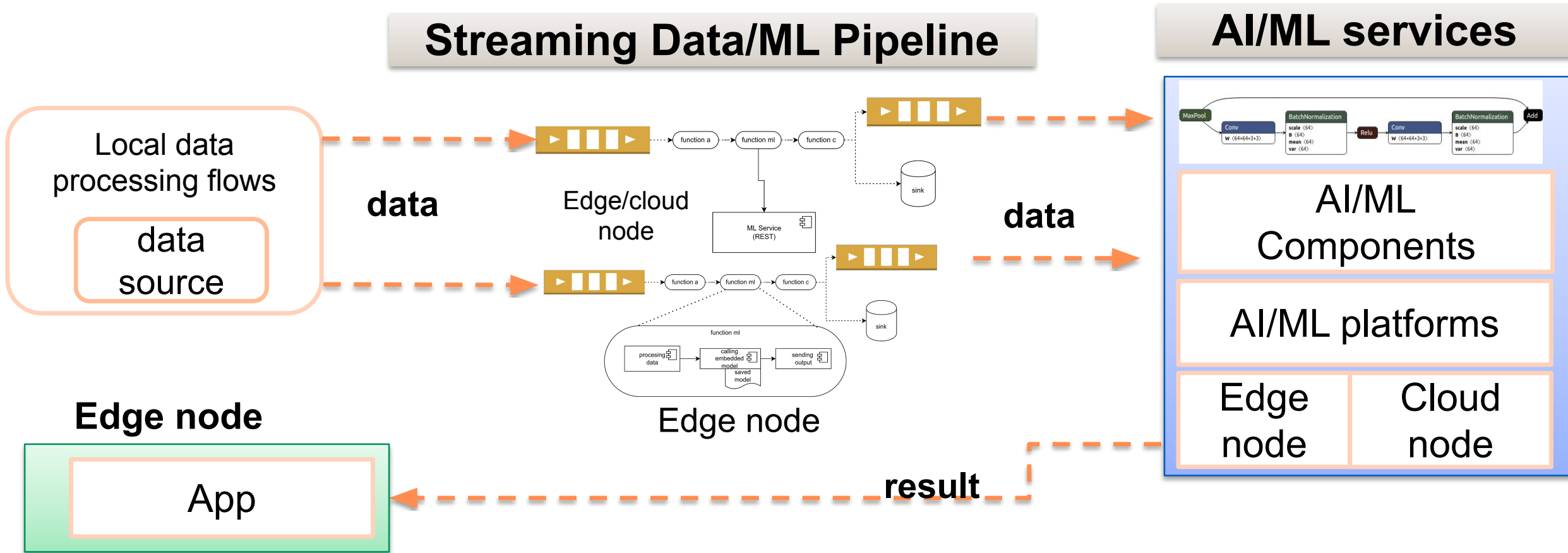


Layers

Software runtime and libraries: not a single software framework/stack

- Many **platform services** according to workloads and architectures, e.g., *messaging systems* with MQTT, NATS, CoAP, AMQP, Kafka, or Pulsar
- Multiple powerful **AI/ML programming frameworks** (PyTorch, TensorFlow, scikit-learn, Keras, Ray, etc.)
- Diverse **AI/ML serving platforms** (Seldon, BentoML, Triton, Ray, ZenML, Lightning LitServe, Ollama, vLLM, etc.)
- Multiple programming models and paradigms: Microservices, serverless, batch workflows, stream processing & reactive system

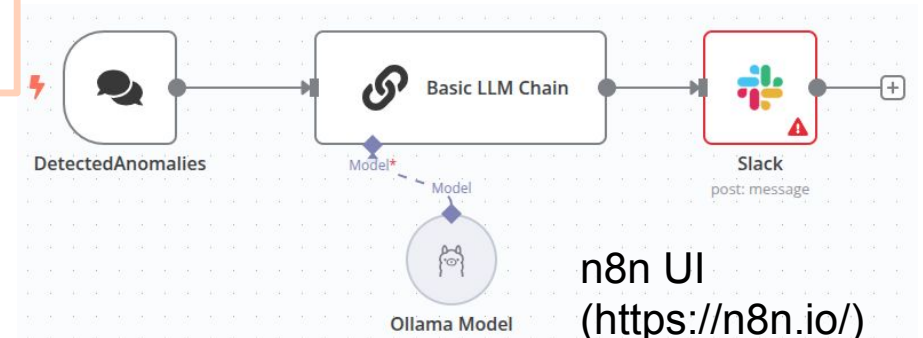
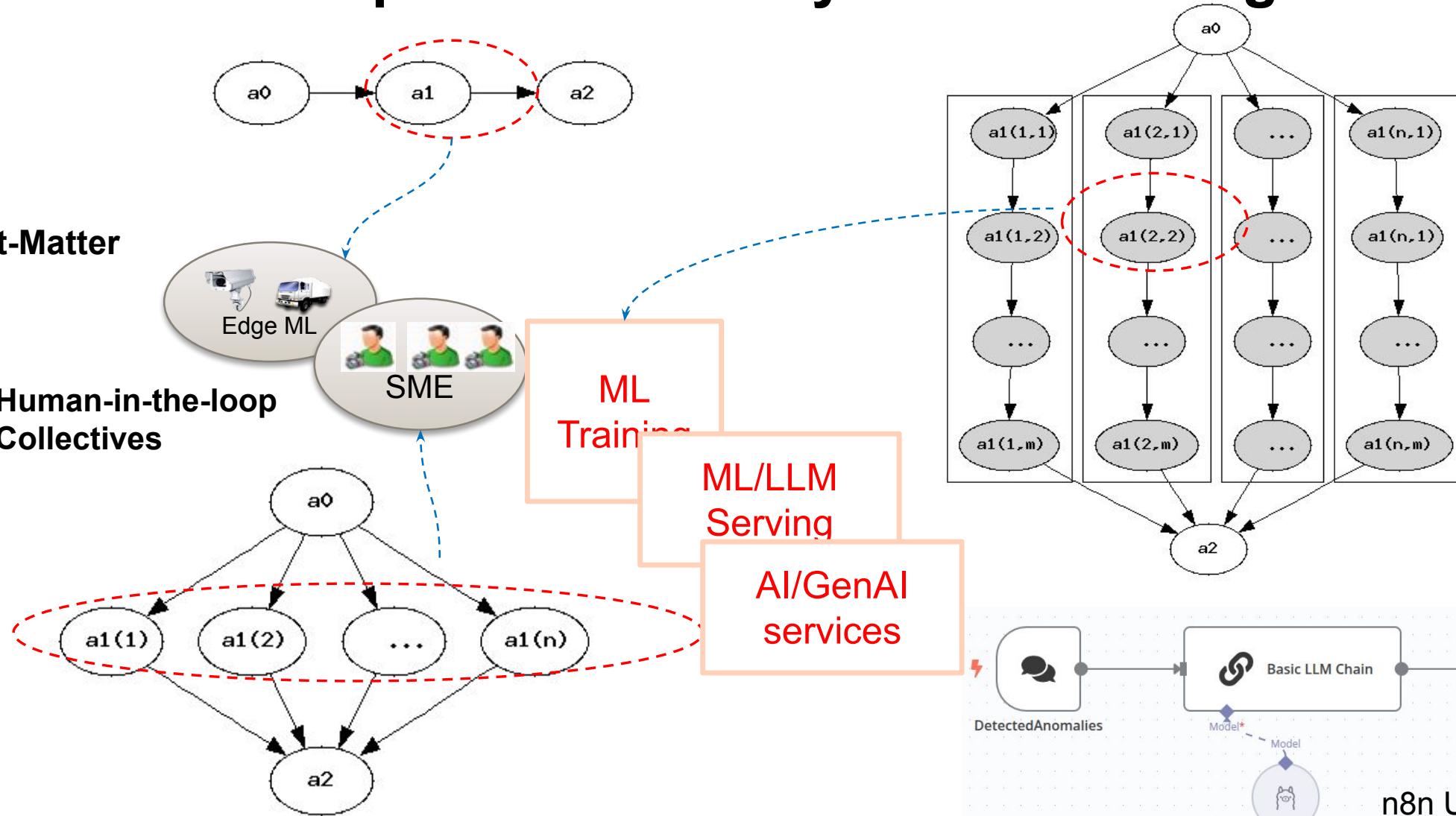
Composition and interactions in edge-cloud applications/systems



Workflows: composition of analytics and intelligence tasks

SME:
Subject-Matter
Expert

Human-in-the-loop
Collectives



Agentic Systems

- Has long history: *“an agent is a computer system situated in some environment, and that is capable of autonomous action in this environment in order to meet its design objectives.”*

(From M. Wooldridge. “An Introduction to Multi-Agent Systems”. Wiley. 2001)

- An AI agent includes many features from monitoring to reasoning to control
- Current trend
 - AI Agents are backed with GenAI/LLMs capabilities

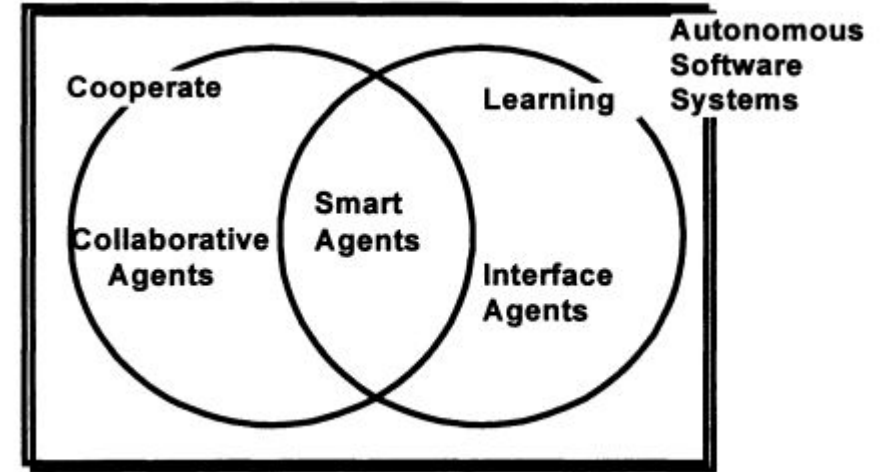
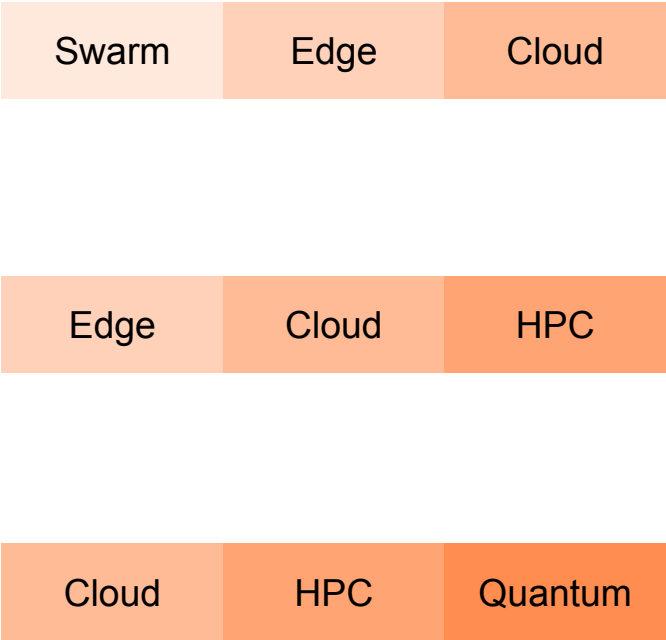
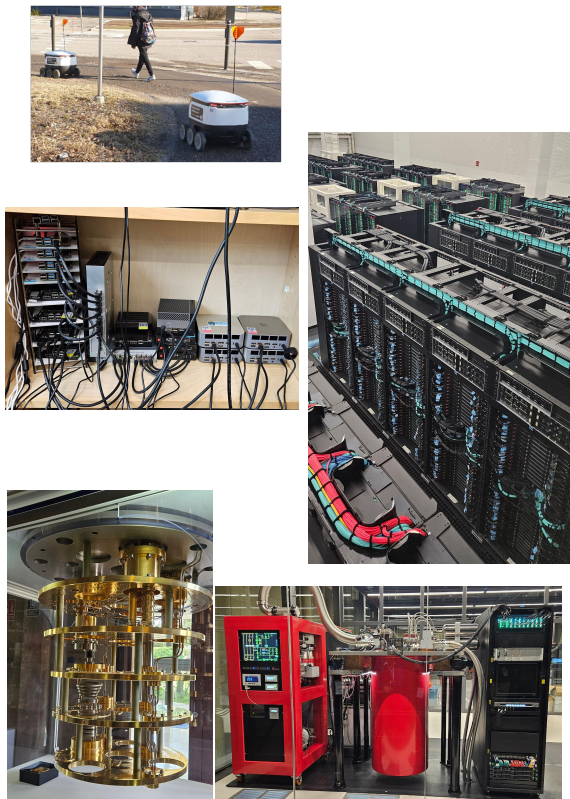


Figure source: Nwana, H.S., Ndumu, D.T. (1998). A Brief Introduction to Software Agent Technology. In: Jennings, N.R., Wooldridge, M.J. (eds) Agent Technology. Springer, Berlin, Heidelberg. https://doi.org/10.1007/978-3-662-03678-5_2

Hybrid Intelligence Software (HIS) in multi-continuum computing

Multi-continuum view

Multiple combinations of computing continuum



Application requirements for multi-continuum

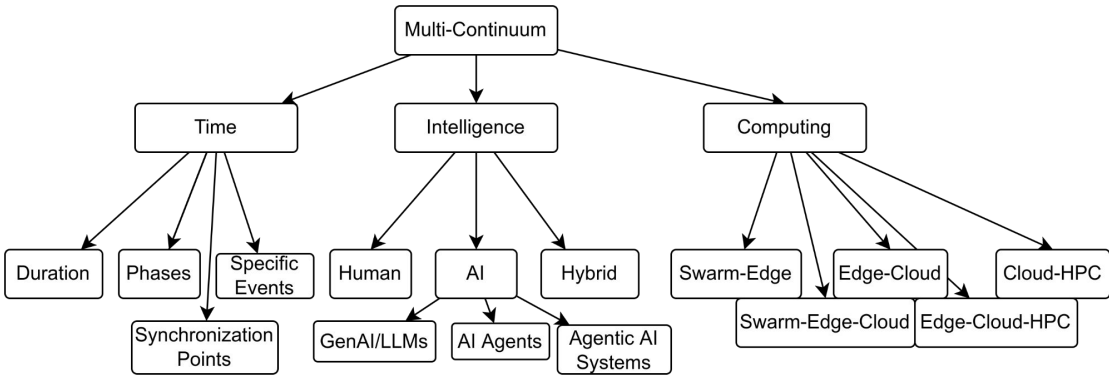
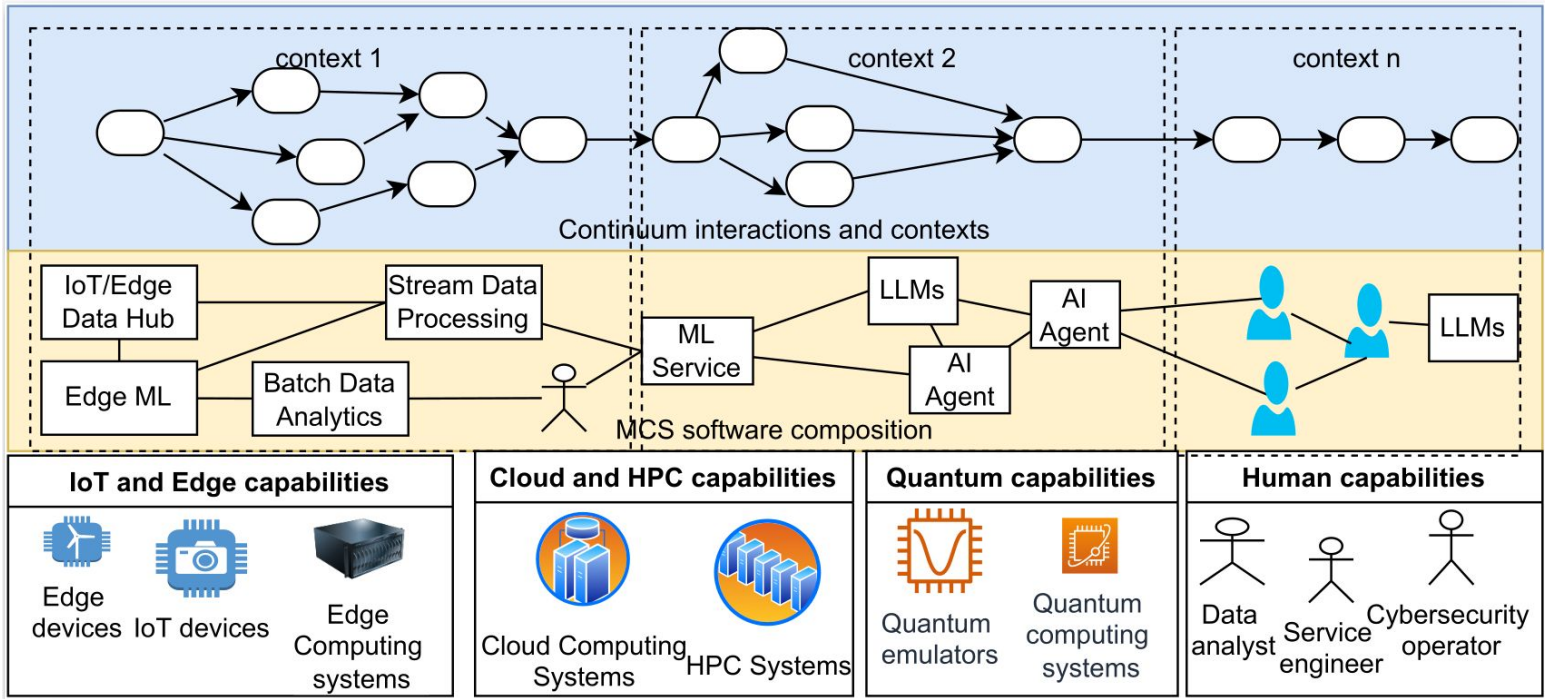


Figure source: Hong-Linh Truong, Kostas Magoutis, “Multi-continuum view for Swarm-Edge-Cloud Service-based Applications”, **working paper**, 2025

Capabilities, composition and context in multi-continuum computing

↑

Capabilities enable complex compositions to solve problems



Context drives the way to compose capabilities to solve problems

↓

Figure source: Hong-Linh Truong, “*New Frontiers in Service Engineering Analytics for Multi-Continuum Systems (FrontSEA)*”, working paper, 2025

Capabilities for solving complex problems: computing, time, and intelligence

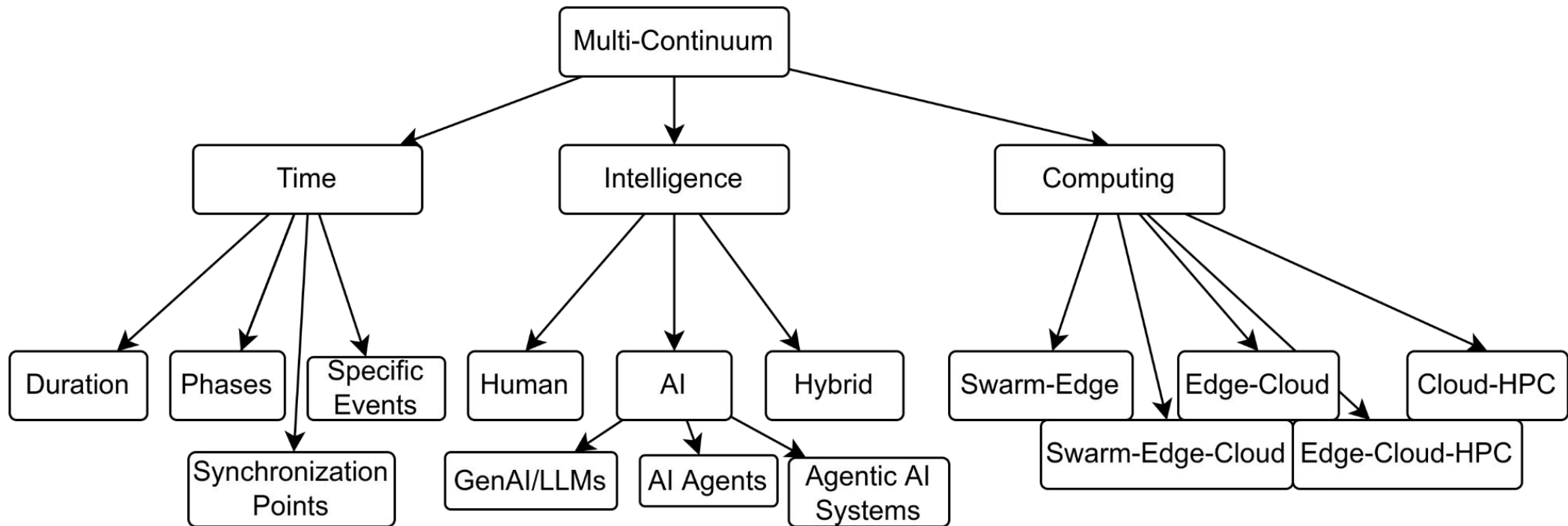


Figure source: Hong-Linh Truong, Kostas Magoutis, "Multi-continuum view for Swarm-Edge-Cloud Service-based Applications", **working paper**, 2025

Hybrid Intelligence Software

- Intelligence capabilities are from different types of services
 - conventional AI/ML, LLM-based services, AI Agents, and humans
- Different compositions of these capabilities for solving complex problems
 - according to different contexts: What, When, Where and How
- Leverage *diversity* of intelligence capabilities
 - implement “intelligence continuum” for solving complex problems (*functional*) to meet quality, *sovereignty* and *compliance* requirements (*non-functional*)

Intelligence continuum: development and operations (1)

- Hybrid intelligence capabilities
 - intelligence capabilities from human can be based on a group of individuals/subject-matter-experts (SMEs)
 - intelligence capabilities from software can be built from ensembles/collectives of GenAI/LLMs/ML services and humans
- Hybrid intelligence can be provided under (within a boundary)
 - a sequence/structured pattern of intelligence capabilities
 - a collective of human and AI capabilities
- Basic operations for capability provisioning
 - add, remove, change, and replace intelligence capabilities
- Continuum state
 - non disrupted intelligence provisioning for consumer/provider requirements

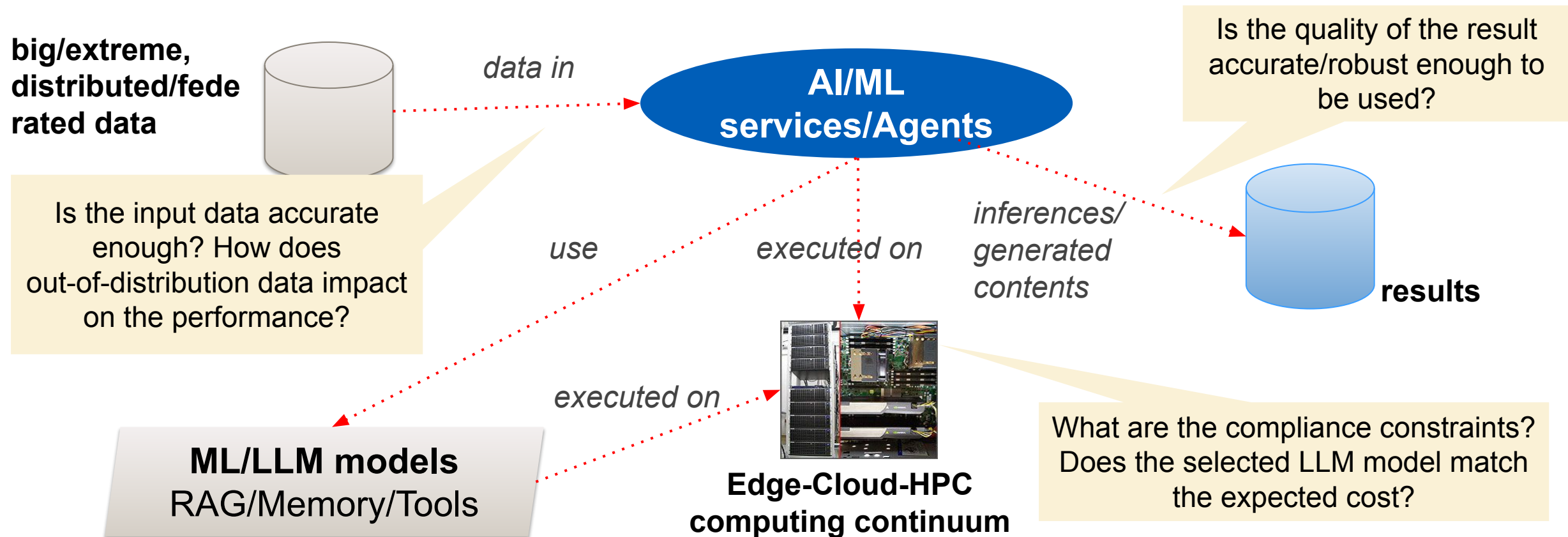
Intelligence continuum: development and operations (2)

- **Which and how GenAI services and AI agents are used in ensembles/collectives?**
 - non-functional properties for GenAI/LLMs are determined and managed based on specific task aspects/views
 - performance, cost, reliability, risk, relevance degrees, etc. based on the type of tasks and contexts
- **Which and how AI+human intelligence capabilities are coordinated for non-functional requirements and continuum requirements?**
 - expected quality attributes: not just time and cost but also, e.g., reliability, risks and sustainability (task and context specific)
- **Which and how quality attributes are linked to risks and the use of guards for AI risks?**
 - complex AI/GenAI risks $\Rightarrow$ trustworthiness
 - many quality attributes cannot be automatically measured
 - combine measurement, feedback and estimation techniques

Hybrid Intelligence Software: non-functional requirements

- Solving complex problems with *non-functional requirements from consumers/providers*
- Quality of analytics (QoA) goals
 - tackle common non-functional requirements w.r.t. availability, performance, cost, etc.
 - so far are well known but challenging
- High-level goals
 - towards more abstract and high level non-functional requirements
 - key emerging dimensions: *diversity, sovereignty, and compliance*

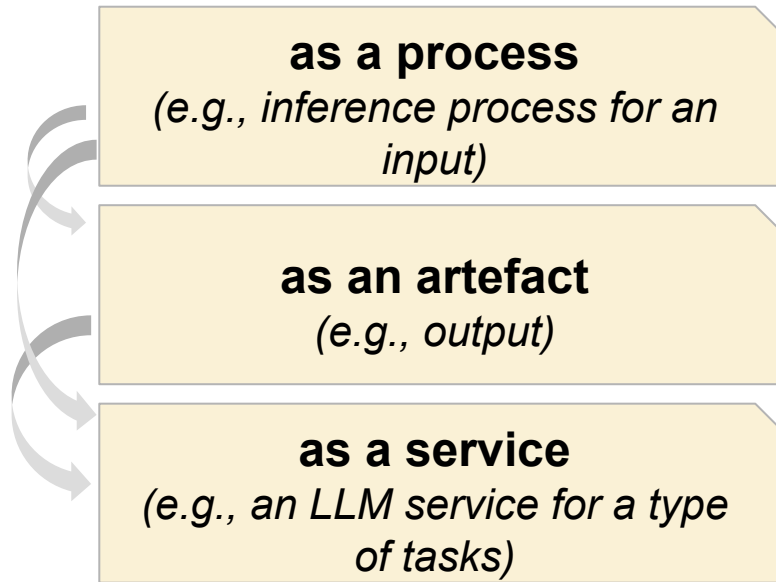
QoA: dependencies across data, AI/ML models, computing resources and quality aspects



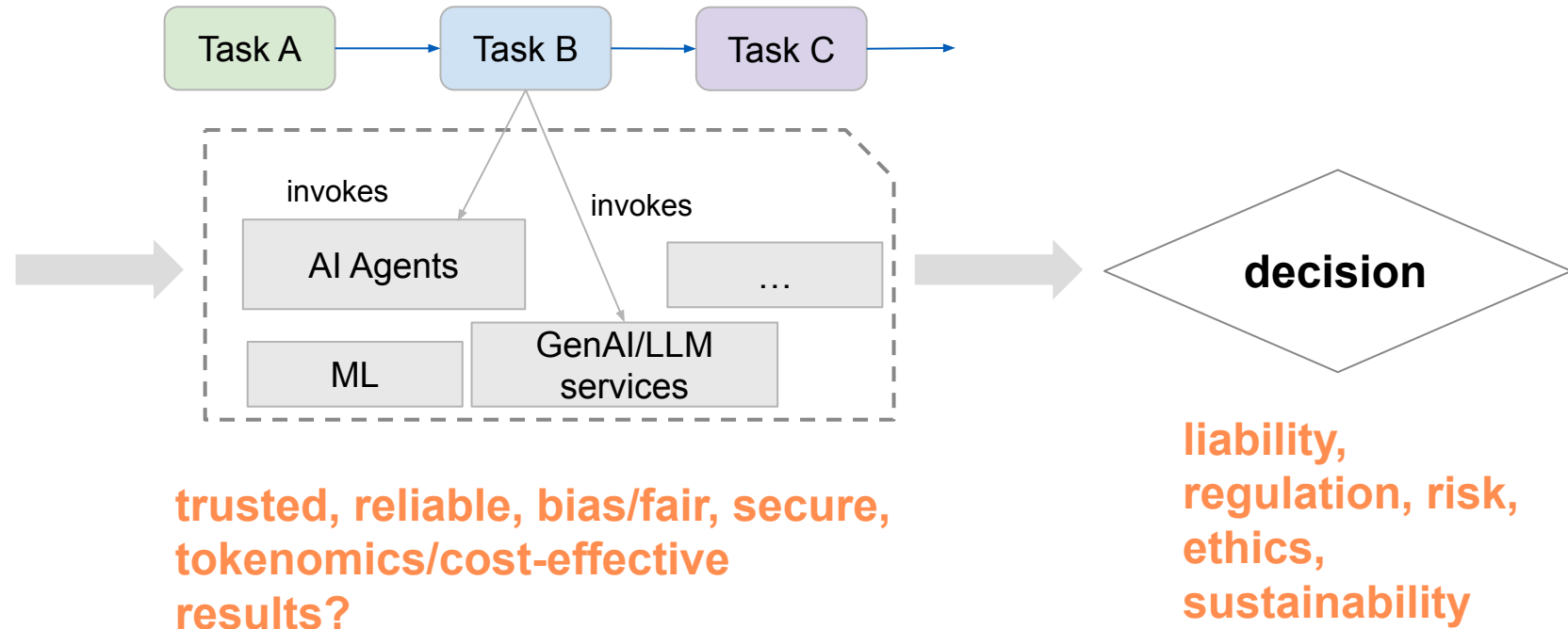
Derived from the Source in: “**Coordination-aware assurance for end-to-end machine learning systems: the R3E approach**”, AI Assurance, <https://doi.org/10.1016/B978-0-32-391919-7.00024-X>

QoA: system views for characterization, observability and analytics experimentation

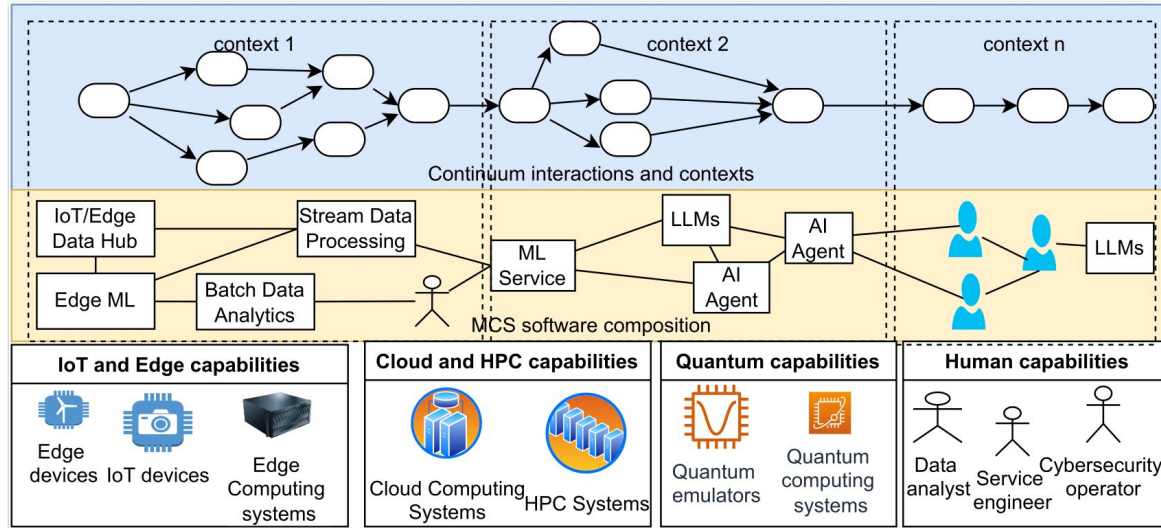
Analytics views



Composition



Hybrid intelligence software: diversity, sovereignty and compliance



built
with/built
for

- Diversity: leverage diverse types of capabilities
- Sovereignty: own and control data/services
- Compliance: satisfy regulations

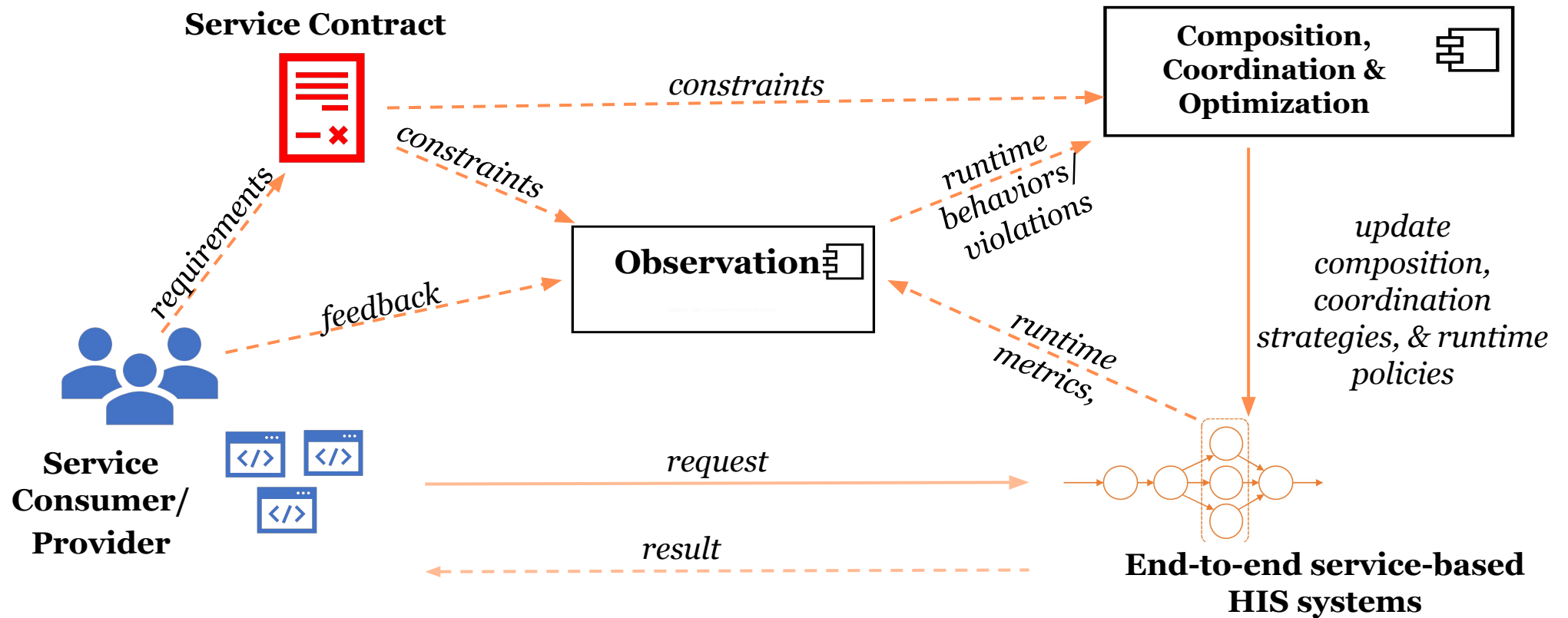
for the rights for service consumers and providers:

- *the right to repair/exchange and compose*
- *the right to own and control*
- *the right to know*
- *the right to deny*
- *the right to forget*

New research for supporting diversity, sovereignty and compliance for hybrid intelligence software

- Service/system characterization
 - various aspects in each dimension and their dependencies
 - links to well-known metrics/QoA views
- Service analytics for development and operations
 - service interactions and patterns (diversity/sovereignty/compliance)
 - resilience, performance and robustness (diversity/compliance)
 - trustworthiness, assurance and risks (sovereignty/compliance)
- New methods/techniques
 - observability and experimentation
 - composition and coordination models
 - runtime policies and adaptation techniques

Operation focus: HIS service contracts, composition and coordination for non-functional requirements



Based on "Novel Contract-based Runtime Explainability Framework for End-to-End Ensemble Machine Learning Serving", CAIN 2024, doi: 10.1145/3644815.3644964

Study log for this week

Think about

- a scenario of service-based HIS in multi-continuum computing
- read one of the papers in the reading list for today lecture

Then

- which kind of HIS characteristics, potentials or challenges do you see?
- ~1-2 page – submit it to the MyCourses for comments/feedback (keep it in your git)

A!

Kiitos
aalto.fi